

Toka Priyanka

tokapriyanka@gmail.com

AI/ML intern

AI/ML Team — Today's Task Focus: 2D & 3D Virtual Try-On Development

Comparison of MediaPipe, OpenPose, Detectron2, and SMPL

Virtual try-on systems rely on accurate human body understanding, pose estimation, segmentation, and 3D body modeling. Several state-of-the-art frameworks are available for these tasks, each serving a different purpose in the pipeline.

MediaPipe

MediaPipe is a lightweight machine learning framework developed by Google that provides real-time body pose estimation and landmark detection. It can identify key body points such as shoulders, elbows, wrists, hips, knees, and ankles with high speed and low computational requirements. MediaPipe is particularly suitable for mobile and web applications because it performs efficiently even on devices with limited processing power. However, its accuracy may decrease when dealing with highly complex poses or severe body occlusions.

OpenPose

OpenPose is a widely used pose estimation framework capable of detecting multiple people and extracting detailed body, face, hand, and foot keypoints. It provides highly accurate skeletal tracking and is commonly used in research and advanced computer vision applications. Although OpenPose delivers excellent results, it requires significantly more computational resources than MediaPipe, making it less suitable for real-time mobile deployment.

Detectron2

Detectron2 is an advanced computer vision framework developed by Meta. It offers powerful object detection, instance segmentation, and keypoint detection capabilities. In virtual try-on systems, Detectron2 is primarily used for accurate body segmentation and human parsing. It can precisely separate a person from the background and identify different body regions, which improves garment fitting quality. The framework achieves high accuracy but generally requires GPU acceleration for optimal performance.

SMPL (Skinned Multi-Person Linear Model)

SMPL is a parametric 3D human body model widely used in 3D avatar generation and virtual try-on applications. It converts body shape and pose information into a realistic 3D human mesh. Unlike MediaPipe, OpenPose, and Detectron2, which mainly operate in 2D space,

SMPL generates a complete 3D representation of the human body. This makes it highly valuable for realistic 3D virtual fitting rooms and avatar-based fashion applications.

Comparative Analysis

MediaPipe offers the fastest performance and is ideal for real-time applications. OpenPose provides the highest pose estimation accuracy but requires greater computational resources. Detectron2 excels in segmentation and body parsing tasks, making it suitable for garment fitting pipelines. SMPL focuses on 3D body reconstruction and enables advanced virtual try-on experiences through realistic avatar generation.

Based on the analysis, a hybrid approach combining Detectron2 for segmentation, MediaPipe for pose estimation, and SMPL for 3D body modeling can provide an effective solution for developing a modern virtual try-on system.

Garment Warping Techniques in Virtual Try-On Systems

Garment warping is one of the most critical components of a virtual try-on system. The objective of garment warping is to transform a clothing image so that it aligns naturally with the user's body shape, pose, and proportions. Simply overlaying a garment onto a user image often produces unrealistic results because clothing must stretch, rotate, bend, and adapt to body movements.

One of the most widely used approaches is Thin Plate Spline (TPS) transformation. TPS performs smooth geometric transformations by mapping control points from the garment image to corresponding points on the user's body. It is computationally efficient and capable of generating natural-looking garment deformations. Many popular virtual try-on systems use TPS as a core component for garment alignment.

Another important technique is the Geometric Matching Module (GMM). GMM learns spatial relationships between garments and human bodies through deep learning. It predicts transformation parameters that allow garments to fit different body shapes and poses more accurately. Compared to traditional geometric transformations, GMM provides improved fitting quality and handles pose variations more effectively.

Flow-based warping methods represent a more advanced approach. These methods utilize neural networks to estimate dense pixel-level correspondences between garments and body regions. As a result, garments can adapt more naturally to folds, wrinkles, and complex body movements. Although flow-based methods achieve highly realistic results, they typically require larger datasets and greater computational resources.

Recent virtual try-on research combines warping networks with generative models to improve visual realism further. These systems not only align garments accurately but also synthesize realistic textures, shadows, and clothing details. Such approaches significantly enhance user experience and create more convincing virtual fitting results.

Overall, TPS remains suitable for lightweight applications, GMM provides a balance between accuracy and efficiency, while flow-based methods offer the highest realism for advanced virtual try-on systems.

3D Avatar Generation Approaches for Virtual Try-On

Three-dimensional avatar generation has become an essential technology for next-generation virtual try-on systems. Unlike traditional 2D methods, 3D avatars provide a complete representation of the user's body, allowing garments to be visualized from multiple viewpoints and under different poses.

One of the most popular approaches is the SMPL model. SMPL generates a realistic 3D human mesh by estimating body shape and pose parameters from images or video sequences. The generated avatar consists of thousands of interconnected vertices that accurately represent human anatomy. Due to its flexibility and realism, SMPL has become the industry standard for many virtual fitting room applications.

Another significant approach is PIFuHD (Pixel-Aligned Implicit Function for High-Resolution 3D Human Digitization). PIFuHD reconstructs detailed 3D human models directly from a single RGB image. The method captures fine-grained geometric details such as clothing contours and body structure, producing highly realistic avatars suitable for immersive virtual experiences.

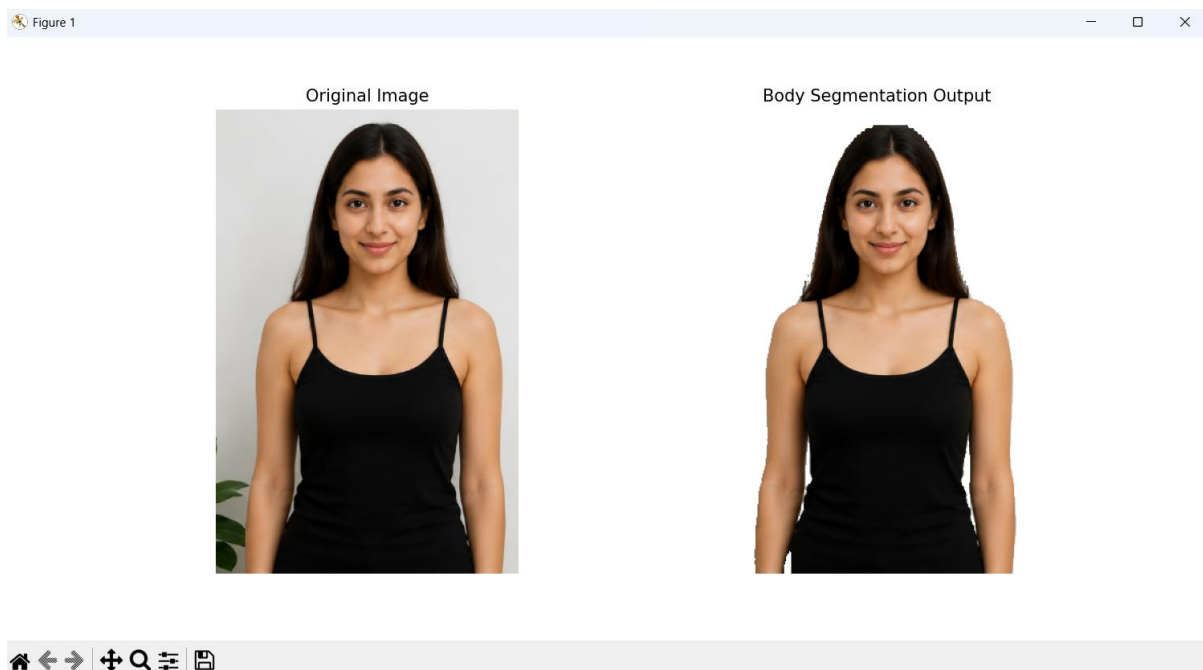
Human Gaussian Splatting is an emerging technology that represents human bodies using millions of Gaussian primitives. This approach enables extremely realistic rendering and smooth visualization from different viewpoints. Gaussian Splatting has recently gained attention because of its ability to generate photo-realistic human representations while maintaining efficient rendering performance.

Neural Radiance Fields (NeRF) also play an important role in 3D human reconstruction. NeRF-based systems learn volumetric representations of people from multiple images and can synthesize highly realistic novel views. Although computationally intensive, NeRF methods produce impressive visual quality and are increasingly being explored for virtual fashion applications.

The future of virtual try-on technology lies in combining accurate body reconstruction, realistic garment simulation, and interactive avatar generation. Among the available approaches, SMPL offers the best balance between accuracy, efficiency, and implementation complexity, making it a strong candidate for developing practical 3D virtual try-on systems for Raritone.

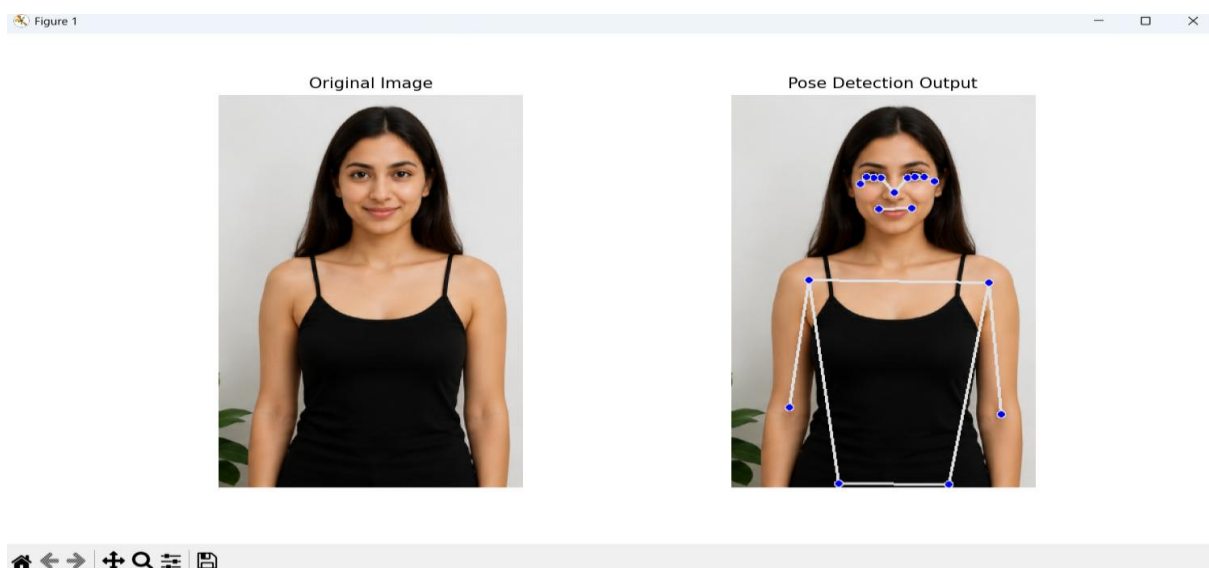
Segmentation outputs

```
dayy > segmentation.py > ...
1  import cv2
2  import mediapipe as mp
3  import matplotlib.pyplot as plt
4  import numpy as np
5  image = cv2.imread("person.jpg")
6  image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
7  mp_segmentation = mp.solutions.selfie_segmentation
8  with mp_segmentation.SelfieSegmentation(model_selection=1) as segmenter:
9      results = segmenter.process(image_rgb)
10 mask = results.segmentation_mask > 0.5
11 background = np.ones_like(image_rgb) * 255
12 output = np.where(mask[:, :, None], image_rgb, background)
13 plt.figure(figsize=(12,6))
14 plt.subplot(1,2,1)
15 plt.imshow(image_rgb)
16 plt.title("Original Image")
17 plt.axis("off")
18 plt.subplot(1,2,2)
19 plt.imshow(output)
20 plt.title("Body Segmentation Output")
21 plt.axis("off")
22 plt.show()
23 cv2.imwrite(
24     "segmentation_output.png",
25     cv2.cvtColor(output.astype(np.uint8), cv2.COLOR_RGB2BGR)
26 )
27 print("Segmentation output saved as segmentation_output.png")
```



Pose detection outputs

```
dayy > pose_detection.py > ...
1  import cv2
2  import mediapipe as mp
3  import matplotlib.pyplot as plt
4  image = cv2.imread("person.jpg")
5  image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
6  mp_pose = mp.solutions.pose
7  mp_draw = mp.solutions.drawing_utils
8  with mp_pose.Pose(
9      static_image_mode=True,
10     model_complexity=2,
11     min_detection_confidence=0.5
12 ) as pose:
13     results = pose.process(image_rgb)
14     output = image_rgb.copy()
15     if results.pose_landmarks:
16         mp_draw.draw_landmarks(
17             output,
18             results.pose_landmarks,
19             mp_pose.POSE_CONNECTIONS
20         )
21     plt.figure(figsize=(12,6))
22     plt.subplot(1,2,1)
23     plt.imshow(image_rgb)
24     plt.title("Original Image")
25     plt.axis("off")
26     plt.subplot(1,2,2)
27     plt.imshow(output)
28     plt.title("Pose Detection Output")
29     plt.axis("off")
30     plt.show()
31     output_bgr = cv2.cvtColor(output, cv2.COLOR_RGB2BGR)
32     cv2.imwrite("pose_detection_output.png", output_bgr)
33     print("Saved: pose_detection_output.png")
```



Garment overlay experiments

```
dayy > garment_overlay.py > ...
1 import cv2
2 import mediapipe as mp
3 import numpy as np
4 import matplotlib.pyplot as plt
5 person = cv2.imread("person.jpg")
6 shirt = cv2.imread("shirt.png")
7 if person is None:
8     print("Error: person.jpg not found")
9     exit()
10 if shirt is None:
11     print("Error: shirt.png not found")
12     exit()
13 shirt_rgb = cv2.cvtColor(shirt, cv2.COLOR_BGR2RGB)
14 lower_white = np.array([220, 220, 220])
15 upper_white = np.array([255, 255, 255])
16 mask = cv2.inRange(
17     shirt_rgb,
18     lower_white,
19     upper_white
20 )
21 shirt_rgba = cv2.cvtColor(
22     shirt_rgb,
23     cv2.COLOR_RGB2RGBA
24 )
25 shirt_rgba[:, :, 3] = 255 - mask
26 shirt = shirt_rgba
27 def crop_transparent(img):
28     alpha = img[:, :, 3]
29     coords = cv2.findNonZero(alpha)
30     if coords is None:
31         return img
32     x, y, w, h = cv2.boundingRect(coords)
33     return img[y:y+h, x:x+w]
34 shirt = crop_transparent(shirt)
35 person_rgb = cv2.cvtColor(
36     person,
37     cv2.COLOR_BGR2RGB
```

```
dayy > garment_overlay.py > ...
38 )
39 h, w = person_rgb.shape[:2]
40 mp_pose = mp.solutions.pose
41 with mp_pose.Pose(
42     static_image_mode=True,
43     model_complexity=2,
44     min_detection_confidence=0.5
45 ) as pose:
46     results = pose.process(person_rgb)
47 if not results.pose_landmarks:
48     print("No pose detected")
49     exit()
50 landmarks = results.pose_landmarks.landmark
51 left_shoulder = landmarks[
52     mp_pose.PoseLandmark.LEFT_SHOULDER
53 ]
54 right_shoulder = landmarks[
55     mp_pose.PoseLandmark.RIGHT_SHOULDER
56 ]
57 left_hip = landmarks[
58     mp_pose.PoseLandmark.LEFT_HIP
59 ]
60 right_hip = landmarks[
61     mp_pose.PoseLandmark.RIGHT_HIP
62 ]
63 left_x = int(left_shoulder.x * w)
64 right_x = int(right_shoulder.x * w)
65 left_y = int(left_shoulder.y * h)
66 right_y = int(right_shoulder.y * h)
67 hip_y = int(
68     ((left_hip.y + right_hip.y) / 2) * h
69 )
70 shoulder_width = abs(
71     right_x - left_x
72 )
73 shirt_width = int(
74     shoulder_width * 1.8
```

```
dayy > garment_overlay.py > ...
75 )
76 aspect_ratio = shirt.shape[0] / shirt.shape[1]
77 shirt_height = int(
78     shirt_width * aspect_ratio
79 )
80 shirt = cv2.resize(
81     shirt,
82     (shirt_width, shirt_height)
83 )
84 center_x = (
85     left_x + right_x
86 ) // 2
87 x = center_x - shirt_width // 2
88 neck_y = min(
89     left_y,
90     right_y
91 )
92 y = neck_y - int(
93     shirt_height * 0.05
94 )
95 x = max(0, x)
96 y = max(0, y)
97 if x + shirt_width > w:
98     shirt_width = w - x
99 if y + shirt_height > h:
100     shirt_height = h - y
101 shirt = cv2.resize(
102     shirt,
103     (shirt_width, shirt_height)
104 )
105 output = person_rgb.copy()
106 alpha = shirt[:, :, 3] / 255.0
107 for c in range(3):
108     output[
109         y:y+shirt_height,
110         x:x+shirt_width,
111         c
```

```
dayy > garment_overlay.py > ...
112     ] = (
113         alpha * shirt[:, :, c]
114         +
115         (1 - alpha) *
116         output[
117             y:y+shirt_height,
118             x:x+shirt_width,
119             c
120         ]
121     )
122 plt.figure(figsize=(15,5))
123 plt.subplot(1,3,1)
124 plt.imshow(person_rgb)
125 plt.title("Original Person")
126 plt.axis("off")
127 plt.subplot(1,3,2)
128 plt.imshow(shirt[:, :, :3])
129 plt.title("Garment")
130 plt.axis("off")
131 plt.subplot(1,3,3)
132 plt.imshow(output)
133 plt.title("Dynamic Virtual Try-On")
134 plt.axis("off")
135 plt.tight_layout()
136 plt.show()
137 cv2.imwrite(
138     "dynamic_tryon_output.png",
139     cv2.cvtColor(
140         output,
141         cv2.COLOR_RGB2BGR
142     )
143 )
144 print("\n===== TRY-ON DETAILS =====")
145 print("Shoulder Width :", shoulder_width)
146 print("Shirt Width :", shirt_width)
147 print("Shirt Height :", shirt_height)
148 print("Position (x,y) :", x, y)
```



Workflow Diagram – 2D Virtual Try-On System

Workflow Description

The user first uploads an image, which undergoes preprocessing operations such as resizing and color conversion. Body segmentation is then performed to isolate the user from the background. The segmented image is passed to a pose detection module, which identifies important body landmarks including shoulders and hips.

The extracted landmarks are used to estimate body dimensions and determine the appropriate garment size and position. The selected garment image is dynamically resized according to the user's body measurements and aligned with the detected pose. Finally, the garment is overlaid onto the user image to generate the virtual try-on result, which is displayed as the final output.

This workflow demonstrates a pose-aware virtual try-on pipeline capable of adapting garment placement according to different body shapes and poses.

System Workflow

